



Progressive Education Society's
Modern College of Engineering, Pune - 411005
Department of Electronics & Telecommunication
Engineering

Program: ECE

AY-2024-25 Semester - I

Database Management System

Course In-charge
Mrs. Supriya Bhosale

UNIT – II

Relational Database Design

Reference: Database System Concepts (6th edition) by Silberschatz A., Korth H., Sudarshan S.

Outline

- Relational Model: Basic concepts, Attributes and Domains
- Relational Integrity: Domain, Referential Integrities
- CODD's Rules
- Features of Good Relational Design
- Normalization
- Atomic Domains and First Normal Form
- Decomposition Using Functional Dependencies
- Functional Dependency Theory

Relational Model

- **Relational model stores data in the form of tables.** This concept proposed by Dr. E. F. Codd, a researcher of IBM in the year 1960s.
- The relational model consists of three major components:
 - The set of relations
 - Set of Integrity rules
 - The operations
- A relational model database is defined as a database that allows you to group its data items into one or more independent tables that can be related to one another by using fields common to each related table.

Basic Structure

- Formally, given sets $D_1, D_2, \dots, D_n$ a **relation** r is a subset of $D_1 \times D_2 \times \dots \times D_n$

Thus a relation is a set of n -tuples $(a_1, a_2, \dots, a_n)$ where each $a_i \subseteq D_i$

- Example: if

customer-name = {Jones, Smith, Curry}

customer-street = {Main, North, Park}

customer-city = {Harrison, Rye, Pittsfield}

Then $r = \{ (Jones, Main, Harrison),$
 $(Smith, North, Rye),$
 $(Curry, North, Rye) \}$

is a relation over *customer-name* $\times$ *customer-street* $\times$ *customer-city*

Characteristics of Relational Database

- Relational database systems have the following characteristics:
 - The whole data is conceptually represented as an orderly arrangement of data into rows and columns, called a **relation or table**.
 - All values are scalar. That is, at any given row/column position in the relation there is one and only one value.
 - All operations are performed on an entire relation and result is an entire relation, **a concept known as closure**.

Basic Terminology used in Relational Model

- The figure shows a relation with the formal names of the basic components marked the entire structure is, as we have said, a relation.

The diagram illustrates a relation as a table. The table has four columns: 'Emp_Code', 'Name', and 'Year'. The first column is labeled 'Attributes' with a horizontal line pointing to it. The first row is labeled 'Tuples' with a vertical line pointing to it. The data rows are as follows:

Attributes	Emp_Code	Name	Year
Tuples	21130	Amar Jain	1
	30745	Kuldeep	3
	41894	Manoj	2
	51207	Rita bajaj	6

- A single row of a table, which contains a single record for that relation is called a **tuple**.. Actually, each row is an n-tuple, but the "n-" is usually dropped.

Basic Terminology used in Relational Model

- **Relation instance** – A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.
- **Relation schema** – A relation schema describes the relation name (table name), attributes, and their names.
 $A_1, A_2, \dots, A_n$ are attributes
 $R = (A_1, A_2, \dots, A_n)$ is a relation schema
E.g. *Customer-schema* = (*customer-name*, *customer-street*, *customer-city*)
- **Cardinality of a relation:** The number of tuples in a relation determines its cardinality. In this case, the relation has a cardinality of 4.
- **Degree of a relation:** Each column in the tuple is called an attribute. The number of attributes in a relation determines its degree. The relation in figure has a degree of 3.
- **Domains:** A domain definition specifies the kind of data represented by the attribute.

Basic Terminology used in Relational Model

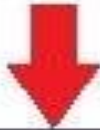
- Keys are very important part of Relational database. They are used to establish and identify relation between tables. They also ensure that each record within a table can be uniquely identified by combination of one or more fields within a table.
- **Super Key –**
Super Key is defined as a set of attributes within a table that uniquely identifies each record within a table. Super Key is a superset of Candidate key.
- **Candidate Key –**
Minimal super key is called as candidate key. Candidate keys are defined as the set of fields from which primary key can be selected. It is an attribute or set of attribute that can act as a primary key for a table to uniquely identify each record in that table.
- **Primary Key**
Primary key is a candidate key that is most appropriate to become main key of the table. It is a key that uniquely identify each record in a table.

Basic Terminology used in Relational Model

■ Primary Key

Primary key is a candidate key that is most appropriate to become main key of the table. It is a key that uniquely identify each record in a table.

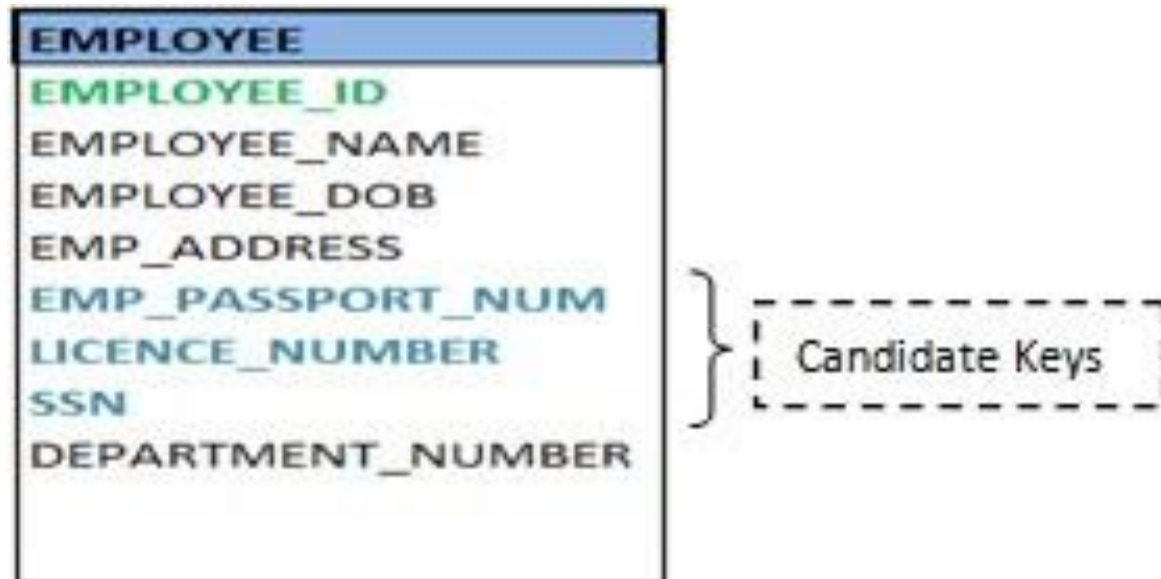
Primary Key



s_id	S_name	age	course	address

Basic Terminology used in Relational Model

- **Candidate Key** -
- **Example** of employee table, EMPLOYEE_ID is best suited for primary key as its from his own employer. Rest of the attributes like passport number, SSN, license Number etc are considered as candidate key.



Basic Terminology used in Relational Model

■ Composite Key

Key that consist of two or more attributes that uniquely identify an entity occurance is called **Composite key**. But any attribute that makes up the **Composite key** is not a simple key in its own.

Composite Key

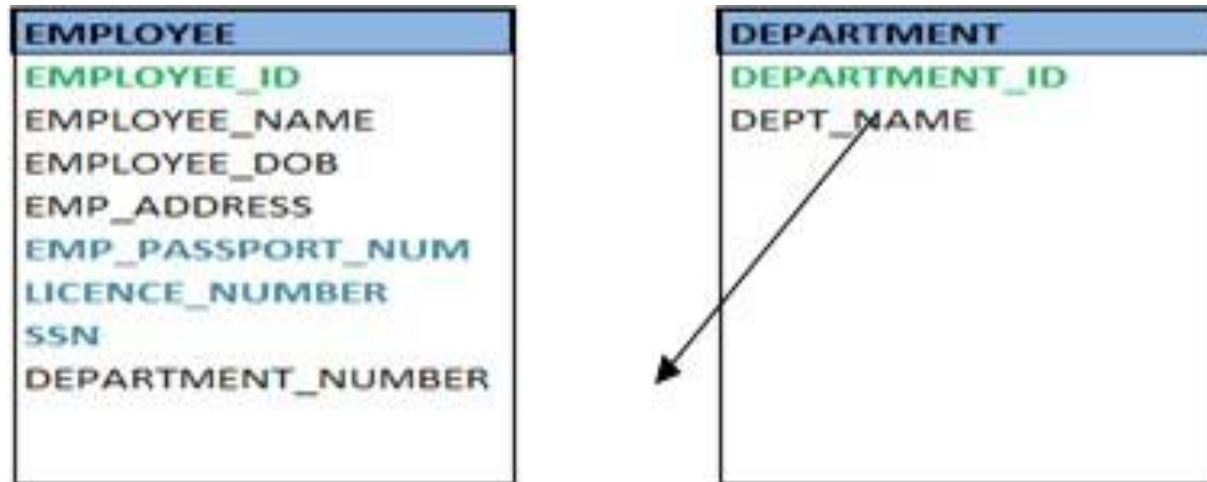


Composite Key		
cust_id	order_id	sale_detail

Basic Terminology used in Relational Model

■ Foreign key -

A **foreign key** is a field (or collection of fields) in one table that uniquely identifies a row of another table or the same table. In simpler words, the **foreign key** is defined in a second table, but it refers to the primary **key** in the first table.



Basic Terminology used in Relational Model

- **Secondary or Alternative key**

The candidate key which are not selected for primary key are known as secondary keys or alternative keys

- **Non-key Attribute**

Non-key attributes are attributes other than candidate key attributes in a table.

- **Non-prime Attribute**

Non-prime Attributes are attributes other than Primary attribute.

Basic Terminology used in Relational Model

- **Constraints-** Every relation has some conditions that must hold for it to be a valid relation. These conditions are called **Relational Integrity Constraints**. There are three main integrity constraints –
 - Key constraints
 - Domain constraints
 - Referential integrity constraints
- 1. **Key Constraints** - There must be at least one minimal subset of attributes in the relation, which can identify a tuple uniquely. This minimal subset of attributes is called key for that relation. If there are more than one such minimal subsets, these are called candidate keys. Key constraints force that –
 - in a relation with a key attribute, no two tuples can have identical values for key attributes.
 - a key attribute can not have NULL values.
- Key constraints are also referred to as Entity Constraints.

Basic Terminology used in Relational Model

2. **Domain Constraints** - Attributes have specific values in real-world scenario. For example, **age can only be a positive integer**. The same constraints have been tried to employ on the attributes of a relation. **Every attribute is bound to have a specific range of values.**

e . g . age cannot be less than zero and

telephone numbers cannot contain a digit outside 0-9.

3. **Referential integrity Constraints** - Referential integrity constraints work on the concept of Foreign Keys. A foreign key is a key attribute of a relation that can be referred in other relation.
- Referential integrity constraint states that **if a relation refers to a key attribute of a different or same relation, then that key element must exist.**
 - **a foreign key must have a matching primary key or it must be null**

Basic Terminology used in Relational Model

- **Example:** Customer table contains the information about the customers with CNO as the primary key. The Customer_Loan table stores the information about CNO, LNO and AMOUNT. It has the primary key combination of CNO and LNO. Here, CNO also acts as the foreign key and refers to CNO of Customer table.

Customer Table			
CNO	NAME	ADDRESS	CITY
C1	Rahal	Thapar campus	Patlala
C2	Ruhi	Tagore Nagar	Jalandhar
C3	Chachat	Dharampura	Qadian
C4	Pooja	GNDU	Amritsar

Customer_Loan Table		
CNO	LNO	AMOUNT
C1	L1	10000
C2	L1	10000
C3	L2	15000
C3	L3	25000
C4	L4	35000

Relationat Model of Custmor-Loan database

Codd's 12 Rules

- Dr Edgar F. Codd, after his extensive research on the Relational Model of database systems, came up with twelve rules (total 13 but starts from 0 till 12 that's why called Codd's 12 Rules).
- **Rule 0: Foundation Rule** - For any system that is advertised as, or claimed to be a Relational Database Management system, that system must be able to manage database entirely through its relational capabilities.
- **Rule 1: Information Rule** - The data stored in a database, may it be user data or metadata, **must be a values of some table cell**. Everything in a database must be stored in a table format.

Codd's 12 Rules

- **Rule 2: Guaranteed Access Rule** - Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value). No other means, such as pointers, can be used to access data.
- **Rule 3: Systematic Treatment of NULL Values** - The NULL values in a database must be given a systematic and uniform treatment. This is a very important rule because a NULL can be interpreted as one the following – data is missing, data is not known, or data is not applicable.

Codd's 12 Rules

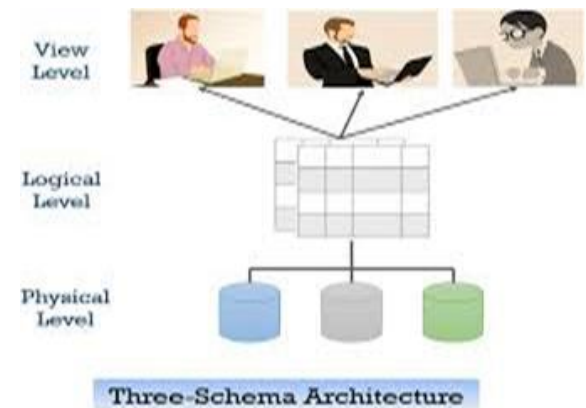
- **Rule 4: Active Online Catalog** - The structure description of the entire database must be stored in an online catalog, known as **data dictionary**, which can be accessed by authorized users. Users can use the same query language to access the catalog which they use to access the database itself.
- **Rule 5: Comprehensive Data Sub-Language Rule** - A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations. This language can be used directly or by means of some application. If the database allows access to data without any help of this language, then it is considered as a violation.

Codd's 12 Rules

- **Rule 6: View Updating Rule** - All the views of a database, which can theoretically be updated, must also be updatable by the system in relation.
- **Rule 7: High-Level Insert, Update, and Delete Rule** - A database must support high-level insertion, updation, and deletion. This must not be limited to a single row, that is, it must also support union, intersection and minus operations to yield sets of data records.

Codd's 12 Rules

- **Rule 8: Physical Data Independence** - The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.
- **Rule 9: Logical Data Independence** - The logical data(structure of table) in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it. For example, if two tables are merged or one is split into two different tables, there should be no impact or change on the user application. This is one of the most difficult rule to apply.



Codd's 12 Rules

- **Rule 10: Integrity Independence** - A database must be independent of the application that uses it. All its integrity constraints can be independently modified without the need of any change in the application. This rule makes a database independent of the front-end application and its interface.
- **Rule 11: Distribution Independence** - The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only. This rule has been regarded as the foundation of distributed database systems.
- **Rule 12: Non-Subversion Rule** - If a system has an interface that provides access to low-level records, then the interface must not be able to subvert the system and bypass security and integrity constraints.

Features of Good Relational Design

classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)

Database schema for the university example.

Combine Schemas?

- Suppose we combine *instructor* and *department* into *inst_dept*
 - (No connection to relationship set *inst_dept*)
- Result is
 1. possible repetition of information and updating in one tuple but not all
 2. can not create new department unless at least one instructor and if we add some how without instructor then ID, name,, salary will take null and PK ID must not be NULL.

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

A Combined Schema Without Repetition

- Consider combining relations

- *sec_class(sec_id, building, room_number)* and
- *section(course_id, sec_id, semester, year)*

into one relation

- *section(course_id, sec_id, semester, year, building, room_number)*

- No repetition in this case

What About Smaller Schemas?

- Suppose we had started with *inst_dept*. How would we know the repetition and need to split up (**decompose**) it into *instructor* and *department*?
- Discovering repetition is **costly**
- Also how would we know that repeated information is just a coincidence or not?
- Write a rule “if there were a schema (*dept_name*, *building*, *budget*), then *dept_name* would be a primary key”
- Denote as a **functional dependency**:
$$dept_name \rightarrow building, budget$$
- In *inst_dept*, because *dept_name* is not a primary key, the building and budget of a department may have to be repeated.
 - This indicates the need to decompose *inst_dept*

What About Smaller Schemas?

- Not all decompositions are good. Suppose we decompose

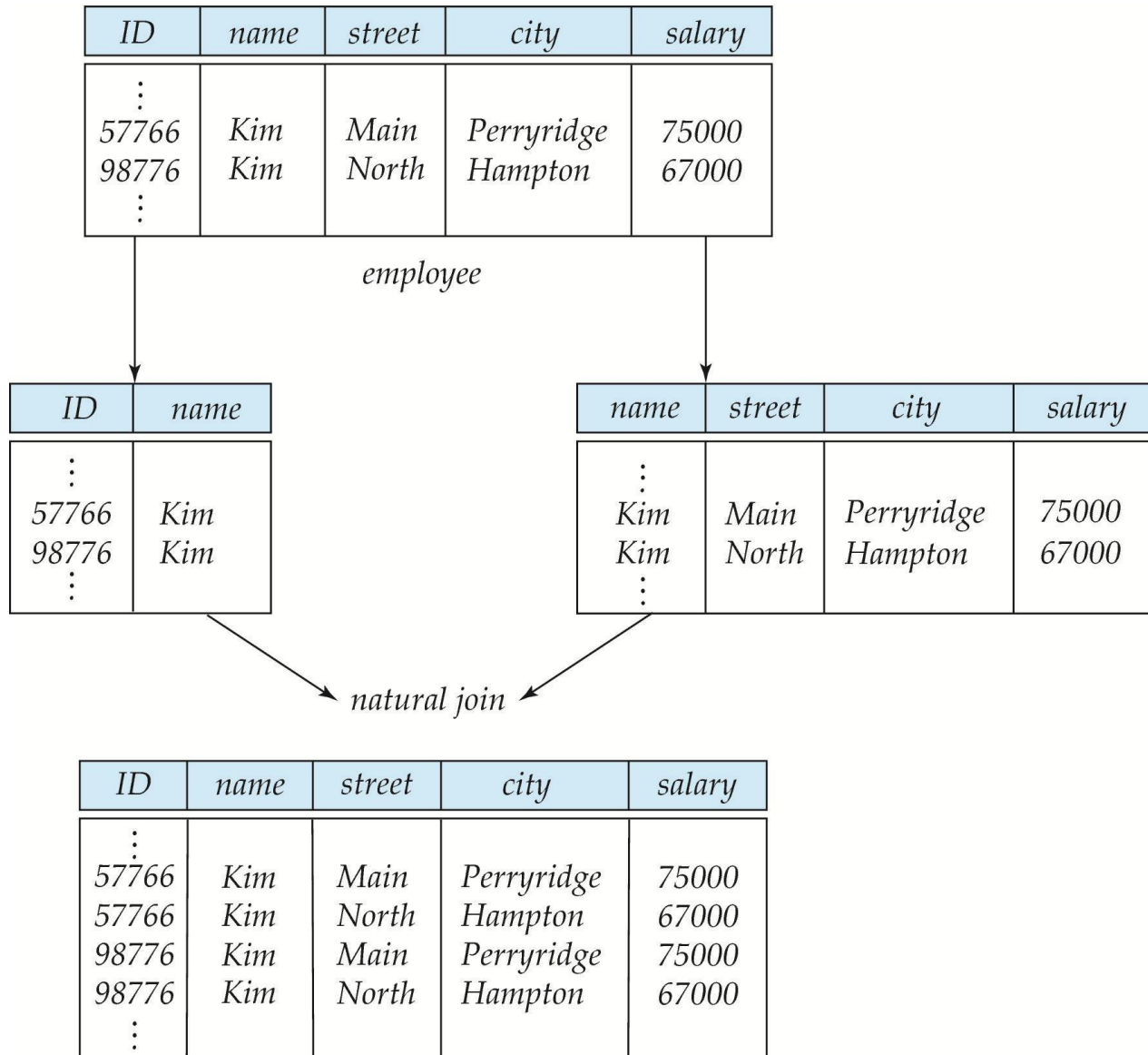
employee(*ID*, *name*, *street*, *city*, *salary*) into

employee1 (*ID*, *name*)

employee2 (*name*, *street*, *city*, *salary*)

- The next slide shows how we lose information -- we cannot reconstruct the original *employee* relation -- and so, this is a **lossy decomposition**.

A Lossy Decomposition



Example of Lossless-Join Decomposition

- Lossless join decomposition
- Decomposition of $R = (A, B, C)$
 $R_1 = (A, B)$ $R_2 = (B, C)$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B

Functional Dependencies

- Relationship that exists between two attributes
 - Represented as FD: $X \rightarrow Y$
 - Left side of FD - determinant,
 - Right side of the production - dependent.
 - **X determines Y or Y is functionally dependent on X.**
 - FD should be found out from relation.
 - Def - FD: $X \rightarrow Y$ is true in relation
 1. $t_1.x \neq t_2.x$ (No two tuple values are same)
No need to check values of Y,
it may or many not be same
 2. If $t_1.x = t_2.x$ (two tuple values are same)
Then $t_1.y = t_2.y$ (y values must be same)

Functional Dependencies (Cont.)

- Let R be a relation schema $X \subseteq R$ and $Y \subseteq R$
- The **functional dependency** $X \rightarrow Y$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes X , they also agree on the attributes Y .

That is, $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$

- Example: Consider $r(A,B)$ with the following instance of r .

A	B
1	4
1	5
3	7

- On this instance, $A \rightarrow B$ does **NOT** hold, but $B \rightarrow A$ does hold.

Functional Dependencies (Cont.)

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
 - $K \rightarrow R$, and
 - for no $\alpha \subset K$, $\alpha \rightarrow R$
- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*).

We expect these functional dependencies to hold:

dept_name $\rightarrow$ *building*

and $ID \rightarrow building$

but would not expect the following to hold:

dept_name $\rightarrow salary$

Student Relation

R. No	Name	Marks	Dept	Course
1	a	78	CS	C1
2	b	60	ECE	C1
3	a	78	CS	C2
4	b	60	ECE	C3
5	c	80	IT	C3
6	f	80	ENTC	C2

Types of FD

■ Trivial

- A functional dependency is **trivial** if it is satisfied by all instances of a relation $X \rightarrow Y$ where, Y is a subset of X . (Attribute determines itself)

In general, $X \rightarrow Y$ is trivial if $Y \subseteq X$

$((X \cap Y) \neq \emptyset)$ ie. Not empty

e. g. (Roll_No, Name) $\rightarrow$ Name

name $\rightarrow$ name

Trivial functional dependency is always valid or true.

■ Non-Trivial

$X \rightarrow Y$ $((X \cap Y) = \emptyset)$

there is nothing common between two attribute. Need to check for valid and invalid conditions

Armstrong Axioms/Inference Rules

- Used to conclude functional dependencies on a relational database.
- The inference rule is a type of assertion.
- It can be applied to a set of FD to derive other FD.
- Using the inference rule, other additional functional dependency can be derived from the initial set.

Armstrong Axioms/Inference Rules

Reflexivity/ reflexive rule

- In the reflexive rule,
- $X \rightarrow Y$ where, Y is a subset of X .
- or
- if Y is a subset of X , then X determines Y
- It is always valid. Or trivial.

Augmentation:

- $X \rightarrow Y$ then $XZ \rightarrow YZ$
- If we augment any other attribute then it is also valid.

Transitivity:

If $(X \rightarrow Y \text{ and } Y \rightarrow Z)$ then $X \rightarrow Z$

Armstrong Axioms/Inference Rules

Union:

- If $(X \rightarrow Y \text{ and } X \rightarrow Z)$ then $X \rightarrow YZ$

Decomposition:

- $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$

Pseudotransitivity:

- If $(X \rightarrow Y \text{ and } WY \rightarrow Z)$ then $WX \rightarrow Z$

Composition:

- If $(X \rightarrow Y \text{ and } Z \rightarrow W)$ then $XZ \rightarrow YW$

Use of Functional Dependencies

- We use functional dependencies to:
 - test relations to see if they are legal under a given set of functional dependencies.
 - ▶ If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F .
 - specify constraints on the set of legal relations
 - ▶ We say that F **holds on** R if all legal relations on R satisfy the set of functional dependencies F .
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances.
 - For example, a specific instance of *instructor* may, by chance, satisfy $name \rightarrow ID$.

Closure of a Set of Functional Dependencies

- Given a set F of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - For example: If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
- The **set of all functional dependencies** logically implied by F is the **closure** of F .
- We denote the *closure* of F by **F^+** .
- F^+ is a superset of F .
- We can find F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**
- These rules are
 - **sound** because they do not generate any incorrect functional dependencies
 - **complete** because for given set of F they allow us to generate **F^+** .

Example

- $R = (A, B, C, G, H, I)$

$F = \{ A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H\}$

- some members of F^+

- $A \rightarrow H$ holds

- ▶ by transitivity from $A \rightarrow B$ and $B \rightarrow H$

- $AG \rightarrow I$

- ▶ by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$

- $CG \rightarrow HI$

- ▶ by union property $CG \rightarrow H$ and $CG \rightarrow I$ then $CG \rightarrow HI$

Procedure to compute F^+

$F^+ = F$

apply the reflexivity rule /* Generates all trivial dependencies */

repeat

 for each functional dependency f in F^+

 apply the augmentation rule on f

 add the resulting functional dependencies to F^+

 for each pair of functional dependencies f_1 and f_2 in F^+

 if f_1 and f_2 can be combined using transitivity

 add the resulting functional dependency to F^+

until F^+ does not change any further

Closure of Attribute Sets

- Given a set of attributes α , define the **closure** of α **under** F (denoted by α^+) as the set of attributes that are functionally determined by α under F
- Algorithm to compute α^+ , the closure of α under F

```
result :=  $\alpha$ ;  
while (changes to result) do  
    for each  $\beta \rightarrow \gamma$  in  $F$  do  
        begin  
            if  $\beta \subseteq \textit{result}$  then result := result  $\cup \gamma$   
        end
```

Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$
- $(AG)^+$
 1. $result = AG$
 2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
 3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
 1. Is AG a super key?
 1. Does $AG \rightarrow R?$ == Is $(AG)^+ \supseteq R$ superset
 2. Is any subset of AG a superkey?
 1. Does $A \rightarrow R?$ == Is $(A)^+ \supseteq R$
 2. Does $G \rightarrow R?$ == Is $(G)^+ \supseteq R$

Canonical Cover

- Sets of functional dependencies may have redundant dependencies that can be inferred from the others
 - For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
 - Parts of a functional dependency may be redundant
 - ▶ E.g.: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$
 - ▶ E.g.: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$
- Intuitively, a canonical cover of F is a “minimal” set of functional dependencies equivalent to F , having no redundant dependencies or redundant parts of dependencies

Canonical Cover

- A **canonical cover** for F is a set of dependencies F_c such that
 - F logically implies all dependencies in F_c , and
 - F_c logically implies all dependencies in F , and
 - No functional dependency in F_c contains an extraneous attribute, and
 - Each left side of functional dependency in F_c is unique.
- To compute a canonical cover for F :
repeat
 - Use the union rule to replace any dependencies in F
 $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$
 - Find a functional dependency $\alpha \rightarrow \beta$ with an
extraneous attribute either in α or in β
/* Note: test for extraneous attributes done using F_c , not F^* */
 - If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$**until** F does not change
- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied

Multivalued Dependencies

- Suppose we record names of children, and phone numbers for instructors:
 - *inst_child*(*ID*, *child_name*)
 - *inst_phone*(*ID*, *phone_number*)
- If we were to combine these schemas to get
 - *inst_info*(*ID*, *child_name*, *phone_number*)
 - Example data:
 - (99999, David, 512-555-1234)
 - (99999, David, 512-555-4321)
 - (99999, William, 512-555-1234)
 - (99999, William, 512-555-4321)
- This relation is in BCNF
 - Why?

MVD (Cont.)

- Tabular representation of $\alpha \twoheadrightarrow \beta$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Example (Cont.)

- In our example:

$ID \twoheadrightarrow child_name$

$ID \twoheadrightarrow phone_number$

- The above formal definition is supposed to formalize the notion that given a particular value of Y (ID) it has associated with it a set of values of Z ($child_name$) and a set of values of W ($phone_number$), and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$
 - Indeed we have (in above notation) $Z_1 = Z_2$
The claim follows.

Use of Multivalued Dependencies

- We use multivalued dependencies in two ways:
 1. To test relations to **determine** whether they are legal under a given set of functional and multivalued dependencies
 2. To specify **constraints** on the set of legal relations. We shall thus concern ourselves *only* with relations that satisfy a given set of functional and multivalued dependencies.
- If a relation r fails to satisfy a given multivalued dependency, we can construct a relations r' that does satisfy the multivalued dependency by adding tuples to r .

NORMALIZATION:

- Major aim of relational database design is to group attributes into relations **to minimize data redundancy** and **reduce file storage space required by base relations**.
- How functional dependencies can be used to group attributes into relations that are in a known normal form.

Normalization is the process of reducing /decomposing a relation into a set of small relations free from data redundancy and ensuring data integrity.

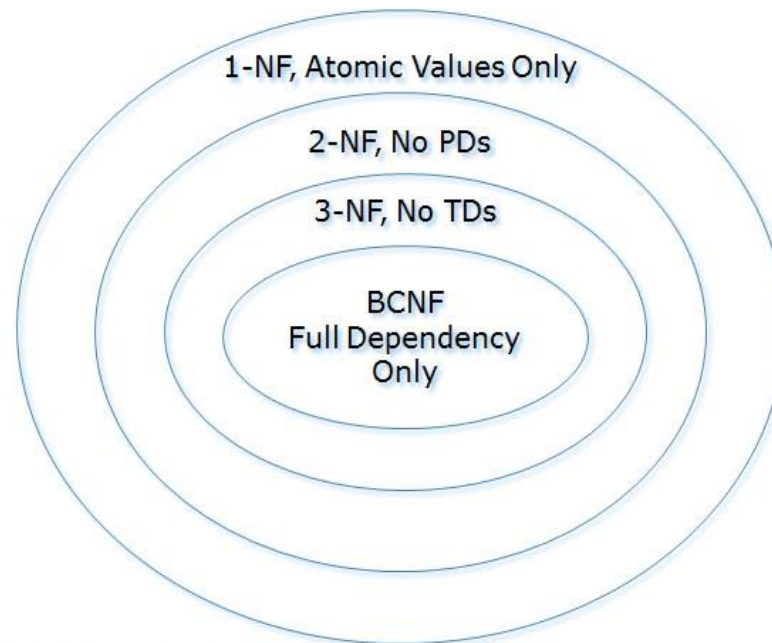
Defined as a step-by-step process of decomposing a complex relation into a simple and stable data structure.

Objectives of Normalization

- To reduce problems associated with redundant data.
- Identification of various types of anomalies such as insertion, deletion, and updation anomalies.
 - I. Insertion anomalies: It may be impossible to store certain information without storing some other, unrelated information.
 - II. Deletion anomalies: It may be impossible to delete certain information without losing some other, unrelated information.
 - III. Update anomalies: If one copy of such repeated data is updated, all copies need to be updated to prevent inconsistency.
- Increasing storage requirements: The storage requirements may increase over time.
- To simplify queries.
- To simplify database structure by decomposition.
- To minimize the NULL values.

Successive normal forms of a relation:

- There are successive higher normal forms such as 1NF, 2NF, 3NF, BCNF, 4NF, 5NF.
- Each normal form is an improvement over the earlier form.
- A higher normal form relation is a subset of lower normal form.



*PD: Partial Dependencies

*TD: Transitive Dependencies

1NF

- The relation is said to be in 1NF if and only if, It contains **no repeating attributes or groups attributes or composite attributes**.
- The relation/table is said to be in 1NF when each cell of the table/relation **contains precisely one value (atomic)**
- There are only **Single Valued Attributes**.
- Attribute Domain does not change.
- There is a unique name for every Attribute/Column.
- The order in which data is stored does not matter.

1NF

SID	S_name	ST_Phone	State	Country
1	Anand	9845763128, 9014723589	MP	India
2	Anand	9879645321	Maharashtra	India
3	Shrikant		Maharashtra	India

Relation STUDENT in table is not in 1NF because of multi-valued attribute ST_PHONE. Its decomposition into 1NF has been shown in next slide

SID	S_name	S_PhoneNo	State	Country
1	Anand	9845763128, 9014723589	MP	India
2	Anand	9879645321	Maharashtra	India
3	Shrikant		Maharashtra	India

SID	S_name	S_PhoneNo	State	Country
1	Anand	9845763128,	MP	India
1	Anand	9014723589	MP	India
2	Anand	9879645321	Maharashtra	India
3	Shrikant		Maharashtra	India



**Conversion
into 1NF**

2NF

A Relation is said to be in Second Normal Form if and only if :

- ***It is in the First normal form, and***
- ***No partial dependency exists between non-prime attributes and prime attributes.***

- Partial dependency: *If the proper subset of candidate key determines non-prime attribute, it is called partial dependency.*

PD = Proper subset of candidate key → Non prime attribute

- **2NF is a fundamental concept of database normalization that helps remove partial dependencies in your relational database.**

e.g. $R(A,B,C,D)$, FD: $\{AB \rightarrow C, BC \rightarrow D\}$

2NF

S_NO	Course_NO	Course_Fee
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

Here,

- COURSE_FEE cannot alone decide the value of COURSE_NO or S_NO;
- COURSE_FEE together with S_NO cannot decide the value of COURSE_NO;
- COURSE_FEE together with COURSE_NO cannot decide the value of S_NO;

Hence,

- COURSE_FEE would be a non-prime attribute, as it does not belong to the one only candidate key {STUD_NO, COURSE_NO} ;

But,

- COURSE_NO → COURSE_FEE, i.e., **COURSE_FEE is dependent on COURSE_NO, which is a proper subset of the candidate key.**
- Non-prime attribute COURSE_FEE is dependent on a proper subset of the candidate key, which is a partial dependency and so this relation is not in 2NF.

2NF

To convert the above relation to 2NF, we need to split the table into two tables such as :

Table 1: S_NO, COURSE_NO

Table 2: COURSE_NO, COURSE_FEE

S_NO	Course_NO
1	C1
2	C2
1	C4
4	C3
4	C1
2	C5

Course_NO	Course_Fee
C1	1000
C2	1500
C4	2000
C3	1000
C5	2000

NOTE: 2NF tries to reduce the redundant data getting stored in memory. For instance, if there are 100 students taking C1 course, we don't need to store its Fee as 1000 for all the 100 records, instead, once we can store it in the second table as the course fee for C1 is 1000.

3NF

A relation R is said to be in the Third Normal Form (3NF) if and only if

- It is in 2NF and*
- No transitive dependency exists between non-prime attributes.*
- If Non Prime attribute $\rightarrow$ Non Prime Attribute then relation is not in 3NF**

A relation is in third normal form if it holds at least one of the following conditions for every non-trivial functional dependency $X \rightarrow Y$.

1. X is a super key. i. e. **LHS is SK.**
2. **Y is a prime attribute (RHS),** i.e., each element of Y is part of some candidate key.

e.g. $R(A,B,C,D)$, FD: $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

3NF

A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.

- **3NF is used to reduce the data duplication. It is also used to achieve the data integrity.**
- **If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.**

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
1	Anand	10009	MP	Jabalpur
2	sujit	10006	Maharashtra	Pune
3	Shrikant	10005	Maharashtra	Mumbai
4	mahi	10007	Rajasthan	Jaipur
5	sanvi	10003	Gujarat	Surat

FD: EMP_ID → EMP_NAME, EMP_ID → EMP_ZIP, EMP_ZIP → EMP_STATE, EMP_CITY

Super key in the table above:

1. {EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID. The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key(EMP_ID). **It violates the rule of third normal form.**

Employee table

EMP_ID	EMP_NAME	EMP_ZIP
1	Anand	10009
2	sujit	10006
3	Shrikant	10005
4	mahi	10007
5	sanvi	10003

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

Employee_zip table

EMP_ZIP	EMP_STATE	EMP_CITY
10009	MP	Jabalpur
10006	Maharashtra	Pune
10005	Maharashtra	Mumbai
10007	Rajasthan	Jaipur
10003	Gujarat	Surat

BCNF

A relation is said to be in Boyce Codd Normal Form (BCNF)

- if and only if all the determinants are candidate keys.

BCNF relation is a strong version of 3NF, but not every 3NF relation is BCNF.

A relation is said to be in BCNF if –

i) It is in 3NF

ii) For each non trivial dependancies, the LHS (determinant) must be a superkey or candidate key.

if $X \rightarrow Y$ then X must be a super key.

e.g. $R(A,B,C)$, FD: $\{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

e.g. $R(A,B,C,D)$, FD: $\{A \rightarrow BCD, BC \rightarrow AD, D \rightarrow B\}$

Example: Let's assume there is a company where employees work in more than one department.

- **FD:** EMP_ID → EMP_COUNTRY
EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}
- **Candidate key:** {EMP-ID, EMP-DEPT}
- The table is not in BCNF because neither EMP_DEPT nor EMP_ID **alone are keys**.
- To convert the given table into BCNF, we decompose it into three tables:

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

1) EMP_COUNTRY table:

EMP_ID	EMP_COUNTRY
264	India
264	India

1) EMP_DEPT table:

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232

1) EMP_DEPT_MAPPING table:

EMP_ID	EMP_DEPT_NO
264	283
264	300
364	232
364	549

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Consider a relation R with attributes (student, subject, Teacher).

Student	Teacher	Subject
Anand	Pradipkumar	DBMS
Anand	Vishal	MCA
Vinay	Pradipkumar	DBMS
Vinay	Kavita	MCA

F: { (student, Teacher) $\rightarrow$ subject , (student, subject) $\rightarrow$ Teacher,
Teacher $\rightarrow$ subject }

- **Candidate Keys** are (student, teacher) and (student, subject).
- The relation is in 3NF [since there is no transitive dependency]. A relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a key.
- **The above relation is not in BCNF, because in the FD (teacher $\rightarrow$ subject), teacher is not a key.** This relation suffers with anomalies –
- For example, if we try to delete the student Vinay, we will lose the information that Kavita teaches MCA. These difficulties are caused by the fact the teacher is determinant but not a candidate key.

Decomposition for BCNF

- Teacher $\rightarrow$ subject violates BCNF [since teacher is not a candidate key].
- If $X \rightarrow Y$ violates BCNF then divide R into $R_1(X, Y)$ and $R_2(R - Y)$.
- So R is divided into two relations $R_1(\text{Teacher, subject})$ and $R_2(\text{student, Teacher})$.

R1

Teacher	Subject
Pradipkumar	DBMS
Vishal	MCA
Kavita	MCA

R2

Student	Teacher
Anand	Pradipkumar
Anand	Vishal
Vinay	Pradipkumar
Vinay	Kavita

Normal Form	Basic Motivation
1NF	Removing non-atomicity
2NF	Removing partial dependency (Proper subset of key attribute $\rightarrow$ Non-key attribute)
3NF	Removing transitive dependency (NPA $\rightarrow$ NPA) Either LHS is SK Or RHS is prime attribute.
BCNF	Removing any kind of redundancy LHS must be SK.